

Обсуждения, статистику и профили авторов этой публикации смотрите по адресу: <https://www.researchgate.net/publication/272494253>

На пути к нечеткой структуре оценки усилий в гибком программном обеспечении

Разработка

Статья в Международном журнале компьютерных наук и информационной безопасности, январь 2015 г.

ЦИТАТЫ

43

ЧИТАЕТ

2,274

3 автора:



Атеф Т. Нур Эль-Дин Раслан

8 ПУБЛИКАЦИЙ 76 ЦИТИРОВАНИЙ

[СМОТРЕТЬ ПРОФИЛЬ](#)



Надь Рамадан

Каирский университет

115 ПУБЛИКАЦИЙ 773 ЦИТИРОВАНИЯ

[СМОТРЕТЬ ПРОФИЛЬ](#)



Хешам А. Хефни

Факультет аспирантуры статистических исследований (ФГССИ)

320 ПУБЛИКАЦИЙ 3594 ЦИТИРОВАНИЙ

[СМОТРЕТЬ ПРОФИЛЬ](#)

На пути к нечеткой структуре усилий

Оценка в гибкой разработке программного обеспечения

Атеф Тайх Раслан

Кафедра вычислительной техники и
Информационные науки, Институт
Статистические исследования и изыскания,
Каирский университет, Египет

Наги Рамадан Дарвиш

Кафедра вычислительной техники и
Информационные науки, Институт
Статистические исследования и изыскания,
Каирский университет, Египет

Хешам Ахмед Хефни

Кафедра вычислительной техники и
Информационные науки, Институт
Статистические исследования и изыскания,
Каирский университет, Египет

Аннотация. Оценка трудозатрат в области разработки программного обеспечения – это процесс прогнозирования объема работ, выраженного в человеко-месяцах, необходимого для разработки программного обеспечения. Большинство существующих методов оценки трудозатрат подходят для традиционных программных проектов. Природа гибких программных проектов отличается от природы традиционных программных проектов, поэтому использование традиционных методов оценки трудозатрат может привести к неточным оценкам. Для гибких программных проектов требуется инновационная фреймворк оценки трудозатрат, обеспечивающая точность оценки. Основное внимание в данной статье уделяется использованию нечеткой логики для повышения точности оценки трудозатрат с использованием пользовательских историй путём характеристики входных параметров с помощью трапецевидных функций принадлежности. В данной статье исследователи предложили фреймворк, основанный на нечеткой логике, который получает нечеткие входные параметры: баллы истории (Story Points, SP), фактор уровня реализации (Replication Level Factor, ILF), коэффициенты трения (FR) и динамические силы (Dynamic Forces, DF), которые обрабатываются в несколько последовательных этапов для получения окончательной оценки трудозатрат. Исследователи разработали предлагаемый фреймворк с помощью MATLAB, чтобы подготовить его к последующим экспериментам с реальными наборами данных.

Ключевые слова: гибкая разработка программного обеспечения, оценка усилий, баллы, нечеткая логика

I. ВВЕДЕНИЕ

Управление программными проектами – это процесс планирования, Организация, подбор персонала, мониторинг, контроль и руководство программным проектом. Жизненный цикл проекта состоит из четырех фаз [9, 14]: Инициация проекта, планирование проекта, выполнение проекта и закрытие проекта. Планирование проекта — важнейший этап жизненного цикла программного проекта, поскольку оно включает в себя множество сложных и необходимых действий, таких как процесс оценки программного обеспечения, включающий определение размера будущего программного продукта, оценку требуемых трудозатрат, разработку предварительных графиков проекта и, наконец, оценку общей стоимости проекта.

Оценка трудозатрат — это процесс прогнозирования объема работ, выраженного количеством человек в месяц, необходимых для разработки или поддержки программных проектов, на основе неполных и неопределенных требований. Оценки трудозатрат могут быть использованы в качестве исходных данных для планирования проектов, планов итераций, бюджетов и анализа инвестиций.

Разработка программного обеспечения – это сложная в умственном плане задача [52]. Следовательно, для достижения высокого качества, надежности и отсутствия ошибок программного обеспечения используются различные методологии разработки программного обеспечения и методы обеспечения качества [18]. В последние годы гибкие методы разработки программного обеспечения привлекли большое внимание в области программной инженерии [32]. Гибкие методологии делают акцент на работающем программном обеспечении как на основном измерении прогресса [18, 4]. Гибкие методы работают с нестабильными и изменчивыми требованиями, используя ряд методов, уделяя особое внимание сотрудничеству между командой и клиентами и поддержке ранней поставки продукта [4]. Традиционные методы оценки усилий требуют модификаций или улучшений, чтобы быть подходящими для гибких методологий разработки программного обеспечения. Поэтому в этой статье предлагается инновационная структура оценки усилий, которая поможет в получении точной оценки. Предлагаемая структура основана на использовании нечеткой логики, SP, ILF, FR и DF.

Остальная часть статьи разделена на семь разделов. Раздел II представляет обзор гибкой разработки программного обеспечения, включающий основные характеристики гибкого процесса разработки программного обеспечения и примеры гибких методов. Раздел III представляет краткое введение в нечеткую логику. Раздел IV представляет обзор литературы, включающий некоторые важные статьи и исследования по оценке трудозатрат. Раздел V представляет методы оценки трудозатрат и фокусируется на пунктах истории. Раздел VI представляет предлагаемую структуру и объясняет основные шаги, необходимые для достижения окончательной оценки трудозатрат с использованием этой структуры. Раздел VII представляет заключение статьи, включая основные идеи, обсуждаемые в статье. Раздел VIII представляет идеи, которые, как ожидается, будут рассмотрены в будущем.

II. Гибкая разработка программного обеспечения

Гибкая разработка программного обеспечения (ASD) — это группа процессов разработки программного обеспечения, которые являются итеративными, инкрементальными, самоорганизующимися и эмерджентными [10]. Кроме того, её можно определить как сочетание гибкости, проворства, готовности к движению, активности, ловкости в движении и адаптивности [17]. Гибкие методологии — это лёгкий, эффективный, малорисковый, гибкий, предсказуемый, научный и увлекательный способ разработки программного обеспечения. В ASD время и ресурсы доступны, что, как правило,

Считается фиксированным [17]. В традиционных методах время и ресурсы являются гибкими, а функциональность считается фиксированной по сравнению с гибкими [17]. В феврале 2001 года был создан Манифест гибкой разработки программного обеспечения, включающий двенадцать принципов гибкой разработки [7]. Существует несколько гибких методов, основанных на идее гибкого манифеста.

Примерами таких методов являются метод разработки динамических систем (DSDM), экстремальное программирование (XP), разработка на основе функций (FDD) и SCRUM [4, 10, 13, 17, 19, 24, 29].

Характеристики процесса ASD включают в себя: модульность на уровне процесса разработки, итеративный с короткими циклами, ограниченный по времени с итеративными циклами, экономичный процесс разработки, адаптивный, инкрементальный, минимизирующий риски, ориентированный на людей, а также совместный и коммуникативный [10,17]. Преимущества гибкой разработки программного обеспечения включают в себя: доход, качество, прозрачность, управление рисками, гибкость, удовлетворенность клиентов и помощь в создании правильного продукта. Доход относится к инкрементальному характеру гибкой разработки, что позволяет реализовать его на ранних этапах, по мере того как продукт продолжает развиваться. Прозрачность относится к поощрению на протяжении всей разработки продукта и к очень кооперативному подходу. Кроме того, в ASD небольшие инкрементальные релизы видны владельцу продукта и команде продукта, что помогает им выявлять любые проблемы на ранних этапах и облегчить реагирование на изменения.

В традиционных методологиях рабочая нагрузка члена команды определяется руководителем, который оценивает, сколько времени займут определенные задачи, а затем распределяет работу, исходя из общего доступного времени каждого члена команды. В гибкой методологии, напротив, работы назначаются всей команде, а не отдельному ее участнику. Кроме того, гибкая методология рассматривает самоорганизацию команды как фактор успеха, который необходимо поощрять. На рисунке 1 показана оценка трудозатрат в скраме. Методология, применяемая на этапе подготовки к игре. На совещании по планированию спринта команда оценивает свои усилия по историям в бэклоге. Команда делится изученными концепциями и масштабами. Владелец продукта эти оценки необходимы для расстановки приоритетов в бэклоге и прогнозирования релизов с учетом скорости работы команды [33].

III. НЕЧЕТКАЯ ЛОГИКА

Нечёткая логика (НЛ) — это методология решения задач, слишком сложных для количественного понимания [5]. Она основана на теории нечётких множеств и предложена в 1965 году Лотфи Заде [1]. Система нечёткой логики работает с нечёткими параметрами, которые учитывают неточность и неопределённость, определяя путь от заданного входа к выходу с помощью вычислительной платформы, называемой системой нечёткого вывода (СНП).

Система нечёткого вывода — это система, основанная на знаниях или правилах. FIS состоит из четырёх компонентов: фаззификатора, базы нечётких правил, механизма нечёткого вывода и дефаззификации. Фаззификатор преобразует чёткие входные данные в нечёткое множество, представляющее собой нетрадиционный тип множеств, позволяющий элементу иметь частичную степень принадлежности. Принадлежность относится к степени включения в определённое множество. Существует множество видов принадлежности.

функции, но в этом исследовании мы будем использовать функцию принадлежности треугольника, представленную уравнением (1) ниже [12]:

$$\text{Треугольник}(x: a, b, c) = \max\left(\min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right), 0\right) \quad (1)$$

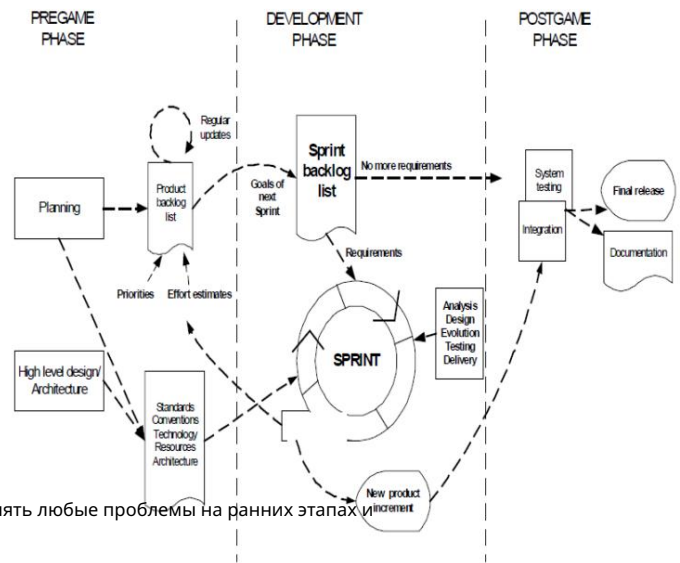
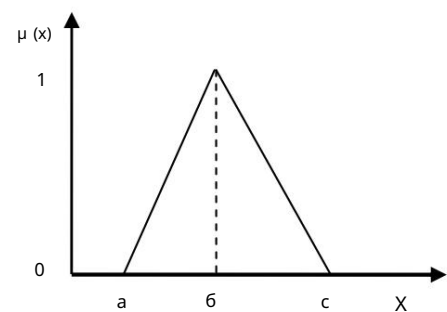


Рисунок 1: Процесс Scrum [29]

На рисунке 2 показана форма треугольной функции принадлежности.



$$\text{Треугольник}(x: a, b, c) = \begin{cases} 0 & x < a \\ \frac{x-a}{b-a} & a \leq x \leq b \\ \frac{c-x}{c-b} & b \leq x \leq c \\ 0 & x > c \end{cases}$$

Рисунок 2: Треугольная функция принадлежности[12]

Механизм нечеткого вывода представляет собой набор правил ЕСЛИ - ТО, хранящихся в базе нечетких правил, называемых механизмом вывода. Дефазификация — это процесс преобразования нечётких выходных данных в чёткие. Наиболее распространённым инструментом для нечётких систем является MATLAB, который используется для определения входных и выходных данных, правил вывода и формы функции принадлежности для нечёткой системы.

Оценка трудозатрат на разработку программного обеспечения — одна из самых сложных задач на этапах разработки проекта [22, 34]. Однако процесс оценки трудозатрат по своей природе неопределён, поскольку в значительной степени зависит от некоторых параметров, которые на ранних этапах разработки неясны. Нечёткая логика использует концепцию нечётких множеств для обработки неточных и неопределённых данных [1].

IV. ОБЗОР ЛИТЕРАТУРЫ

Существует множество статей и исследований по оценке усилий. Ниже приведены некоторые примеры такой литературы:

Масатеру Т. и др. представили пересмотр оценки трудозатрат на разработку программного обеспечения на основе данных о деятельности на ранних этапах разработки. Целью данного исследования является изучение взаимосвязи между трудозатратами на ранних этапах разработки и размером программного обеспечения. Предлагаемая модель была построена с использованием данных о трудозатратах на ранних этапах разработки и размере программного обеспечения. Эта модель была оценена с использованием набора данных Международной группы по стандартам бенчмаркинга программного обеспечения (ISBSG), собранного среди компаний-разработчиков программного обеспечения. Результаты эксперимента показали, что при использовании в качестве объясняющих переменных размера программного обеспечения и суммарных затрат на этапы планирования и анализа требований точность оценки была наиболее высокой [23].

Андреас С. и др. представили руководство по оценке трудозатрат при гибкой разработке программного обеспечения. Данное исследование посвящено изучению возможностей оценки, особенно для XP. Основное внимание уделяется характеристикам гибких методологий. Данное исследование предоставляет рекомендации по аспектам измерения в рамках XP-проекта. Предлагаемое руководство было оценено с помощью опроса, состоящего из 17 вопросов. Каждый вопрос отражает гибкие методологии и оценку трудозатрат. Опрос показывает, что преимущества гибких методов в настоящее время сложно оценить. В частности, необходимо учитывать затраты на проекты по сопровождению [3].

Амрита Р. и др. представили руководство по оптимизации затрат и трудозатрат в проектах Scrum с использованием функциональной точки и модели COCOMO. Предлагаемое руководство использовалось на реальном примере Xsset для оценки. Кроме того, в нём показаны различные инструменты отслеживания и представления, используемые для оптимизации производительности, такие как график выполнения, JIRA, SVN и SCTM. В результате исследования был сделан вывод о высокой эффективности гибкой методологии разработки программного обеспечения. Кроме того, они пришли к выводу, что путем объединения двух оценок

Модели могут повысить эффективность приложенных усилий и снизить затраты [2].

Шахрух З. и др. предложили модель оценки трудозатрат для гибкой разработки программного обеспечения, учитывающую большинство характеристик гибкой методологии, особенно адаптацию и итерации, где основой для оценки служат истории пользователей. Модель была оценена с использованием эмпирических данных, собранных в ходе 21 программного проекта.

Экспериментальные результаты показывают, что модель обладает хорошей точностью оценки с точки зрения средней величины относительной погрешности (MMRE) и вероятности того, что относительная погрешность проекта будет меньше или равна L PRED (L). MMRE и PRED рассчитываются на основе относительной погрешности, которая представляет собой относительную величину разницы между фактическим и оценочным значением индивидуальных усилий [33].

Зиауддин А. и др. представили модель оценки стоимости программного обеспечения на основе нечеткой логики. Целью данного исследования является использование модели нечеткой логики для повышения точности оценки трудозатрат на разработку программного обеспечения. Основная идея данного исследования заключается в фазификации входных параметров модели COCOMO II и дефазификации результата для получения результирующего усилия. Для представления лингвистических терминов в модели COCOMO II используются треугольные нечеткие числа. Результаты данной модели сравниваются с COCOMO II и моделью Алаа Шета. Предложенная модель дает лучшие результаты с точки зрения MMRE, PRED(n) и счета дисперсии (VAF). VAF используется в контексте статистических моделей, основной целью которых является прогнозирование будущих результатов на основе другой связанной информации [34].

Вишал С. и др. представили оптимизированную структуру на основе нечеткой логики для оценки усилий при разработке программного обеспечения. Производительность предлагаемой платформы продемонстрирована с помощью эмпирической проверки, проведённой на данных реальных проектов из общедоступной базы данных COCOMO. Производительность платформы оценивается с использованием данных реальных проектов из общедоступной базы данных COCOMO. Показано, что предлагаемая платформа может быть развернута в среде COCOMO II с предоставлением экспертами необходимой информации для разработки нечётких множеств и соответствующей базы правил [30].

Ратнеш Л. и Абхай К. представили модель оценки для гибких веб-разработок, направленную на решение многочисленных открытых вопросов веб-разработки, особенно в контексте гибкой разработки программного обеспечения, путем анализа данных завершенных гибких веб-проектов. В этом исследовании определяются различия между традиционными и веб-проектами. Кроме того, предлагается инструмент для оценки трудозатрат на основе гибких манифестов и веб-характеристик [26].

Абир Х. представил модель, основанную на нечеткой логике, для повышения чувствительности модели затрат COCOMO. Эта модель

Исследование повышает точность и чувствительность промежуточного варианта COSOMO 81 за счёт нечёткой модели, учитывающей факторы затрат. Эта модель была реализована в среде MATLAB. Набор данных был собран в шести центрах NASA и охватывает широкий спектр областей программного обеспечения, процессов разработки, языков и уровней сложности, а также фундаментальные различия в культуре и деловой практике каждого центра. Результаты показали, что чувствительность предлагаемой нечёткой модели превосходит промежуточный вариант COSOMO 81 [1].

Джитендер Чоудхурия и Уграсен Суман представили модель оценки затрат на обслуживание программного обеспечения (SMEEM) для оценки затрат на обслуживание программного обеспечения. Предлагаемая модель использует SP для расчета объема затрат на обслуживание и коэффициентов корректировки стоимости, влияющих на количество баллов для оценки затрат. Предлагаемая модель проиллюстрирована на примере различных типов проектов по обслуживанию, таких как веб-приложения, MIS и критически важные проекты. Она разработана, чтобы помочь менеджеру проекта, отвечающему за обслуживание программного обеспечения, рассчитать предполагаемые затраты на обслуживание программного обеспечения с точки зрения скорректированных баллов (ASP), размера, стоимости и продолжительности. Она генерирует более реалистичные и точные результаты оценки. Эта модель применима только для среды обслуживания, основанной на гибком и экстремальном программировании [16].

V. МЕТОДЫ ОЦЕНКИ УСИЛИЙ

Методы оценки трудозатрат на разработку программного обеспечения можно разделить на алгоритмические и неалгоритмические модели [26]. Наиболее популярными моделями, относящимися к неалгоритмическим методам, являются:

- Экспертное заключение.
- Правило большого пальца.
- Метод Дельфи.
- Широкополосный метод Дельфи.
- Закон Паркинсона.
- Выигрышное ценообразование.
- Процесс программного обеспечения зонда.
- Планирование методики командного процесса разработки программного обеспечения.

Недавние исследования были сосредоточены на алгоритмической модели, такой как [6, 20, 26]:

- Строка кода (LOC).
- Функции точечных матриц.
- COSOMO и COSOMO-II.
- Точки объекта.
- Управление жизненным циклом программного обеспечения.
- Оценка вариантов использования.
- Очки истории [8,11,33].

Алгоритмические модели основаны на статистическом анализе исторических данных. Эти модели требуют точного ввода таких атрибутов, как длина строки кода, количество функциональных точек (FP), количество пользовательских экранов, сложность и скорость [8, 33].

В данной статье исследователи сосредоточились на методе Story Point для оценки трудозатрат при гибкой разработке программного обеспечения. Story Point — наиболее распространённый метод оценки для гибкой разработки программного обеспечения [8, 11, 33].

Очки за историю относятся к оценке относительного масштаба работы с точки зрения фактических затрат на разработку. Очки за историю обычно выражаются в числах, следующих за рядом Фибоначчи, размерах футболок (XS, S, M, L, XL, XXL) или даже породах собак (от чихуахуа до немецкого дога) [33]. Ряд Фибоначчи — это последовательность чисел, где каждое число является суммой двух предыдущих. Числа Фибоначчи: 1, 2, 3, 5, 8, 13, 21 и т. д.

Оценка количества историй в пунктах выполняется с помощью относительного размера, путем сравнения одной истории с выборкой историй с ранее оцененным размером. Относительный размер историй, как правило, гораздо точнее на большой выборке, чем попытка оценить каждую историю отдельно с точки зрения затраченных усилий. Покер планирования — один из самых полезных инструментов в гибкой разработке программного обеспечения [31].

В ASD каждому члену команды выдаётся набор карточек. Каждая карточка соответствует определённому размеру в последовательности пользовательских историй. Затем каждый член команды выбирает карту, отражающую его оценку усилий, необходимых для завершения конкретной пользовательской истории, и кладет эту карту перед собой лицевой стороной вниз.

Оценка усилий по гибкой методологии включает размер истории, сложность, фактор уровня реализации и скорость [8]. Размер истории относится к оценке относительного масштаба работы с точки зрения фактических трудозатрат на разработку. Присваиваемые значения обычно берутся из последовательности Фибоначчи. Присваиваемые значения могут быть изменены самой командой, или даже критерии могут быть переопределены. Шкала сложности (Com) назначается аналогично размеру истории с использованием последовательности Фибоначчи. Пользовательские истории группируются в соответствии с характеристиками гибких методов [21]. Кроме того, эти группы могут быть скорректированы самой командой [16].

Скорость вычислений определяет, какой объём работы по продукту команда может выполнить за единицу времени. Фактическая скорость любого проекта разработки зависит от размера и опыта команды [34]. Скорость вычислений помогает команде улучшить свои оценки на протяжении всего жизненного цикла проекта. Кроме того, скорость вычислений определяет производительность команды, что позволяет планировать выпуск и проводить расчёты с течением времени.

Процесс оптимизации подразумевает изучение ограничений проекта, которое необходимо выполнить до калибровки для повышения стабильности расчёта скорости. Этот процесс включает два фактора: FR и DF. Трение включает ряд факторов, которые могут повлиять на скорость команды: состав команды, процесс, факторы окружающей среды и динамика команды [15]. Состав команды зависит от навыков.

и отношение членов команды. Фактор процесса относится к проценту изменений в гибких методах, релизе, сборке и тестировании. Факторы окружающей среды включают шум, плохую вентиляцию, плохое освещение, неудобные сиденья и столы, неподходящее оборудование или программное обеспечение. Командная динамика — это модели взаимодействия между членами команды, которые определяют её эффективность.

В таблице 1 представлен диапазон значений FR. Каждый фактор скорректирован в соответствии с уровнем риска (стабильный, волатильный, высоковолатильный и очень высоковолатильный).

ТАБЛИЦА 1: ФАКТОРЫ ТРЕНИЯ [33]

Факторы трения	Стабильные	летучие	Очень волатильный	Очень высокая волатильность
Состав команды	1	.98	.95	.91
Процесс	1	.98	.94	.89
Факторы окружающей среды	1	.99	.98	.96
Динамика команды	1	.98	.91	.85

DF относится к факторам, которые могут привести к потере скорости [33]. Эти факторы непредсказуемы и неожиданны. DF включает девять факторов: изменения в команде, новые инструменты, дефекты поставщика, обязанности вне проекта, личные проблемы, заинтересованные стороны, нечеткие требования, меняющиеся требования и перераспределение. Эти факторы ранжируются по уровню (нормальный, высокий, очень высокий и сверхвысокий). В таблице 2 представлены присвоенные значения для этих факторов.

ТАБЛИЦА 2: ПЕРЕМЕННЫЕ ФАКТОРЫ [33]

Переменные факторы	Нормальный	Высокий	Очень Высокий	Экстра высокий
Ожидается смена команды	1	.98	.95	.91
Введение в новые инструменты	1	.99	.97	.96
Дефект поставщика	1	.98	.94	.90
Обязанности члена команды вне проекта	1	.99	.98	.98
Личные проблемы	1	.99	.99	.98
Ожидаемая задержка в Реакция заинтересованных сторон	1	.99	.98	.96
Ожидаемая неоднозначность в Подробности	1	.98	.97	.95
Ожидаемые изменения в окружающей среде	1	.99	.98	.97
Ожидаемый переезд	1	.99	.99	.98

VI. ПРЕДЛАГАЕМАЯ СТРУКТУРА

Процесс оценки трудозатрат по своей природе неопределён, поскольку в значительной степени зависит от некоторых атрибутов, которые на ранних этапах разработки остаются неясными [26]. Нечёткая логика использует концепцию нечётких множеств для работы с неопределёнными и неточными данными. Она решает проблемы неточности и неопределённости, определяя путь от заданного входа к выходу с помощью вычислительной платформы, называемой FIS. Она состоит из четырёх основных компонентов: фаззификатора, базы нечётких правил, механизма нечёткого вывода и дефаззификатора. Фаззификатор преобразует чёткие входные данные в нечёткое множество, используя функцию принадлежности. Нечёткая база правил, представленная правилами типа «если-то».

Механизм вывода представляет собой набор правил «если-то».

Дефаззификация — это процесс перевода нечётких данных в чёткие. Для нечётких вычислений использовалась система нечёткого вывода MATLAB.

На рисунке 3 показано, что блок-схема предлагаемой структуры начинается с входных данных SP, ILF, FR и DF. Затем эти входные данные подвергаются фаззификации для получения нечётких множеств с использованием треугольной функции-члена. База правил включает группу правил типа «ЕСЛИ-ТО», которые определяют нечёткие переменные сложности и скорости. Система FIS оценивает все правила базы правил. Полученное нечёткое множество дефаззифицируется до чёткого значения, после чего вычисляются сложность и скорость. Наконец, трудозатраты оцениваются с использованием полученных значений сложности и скорости.

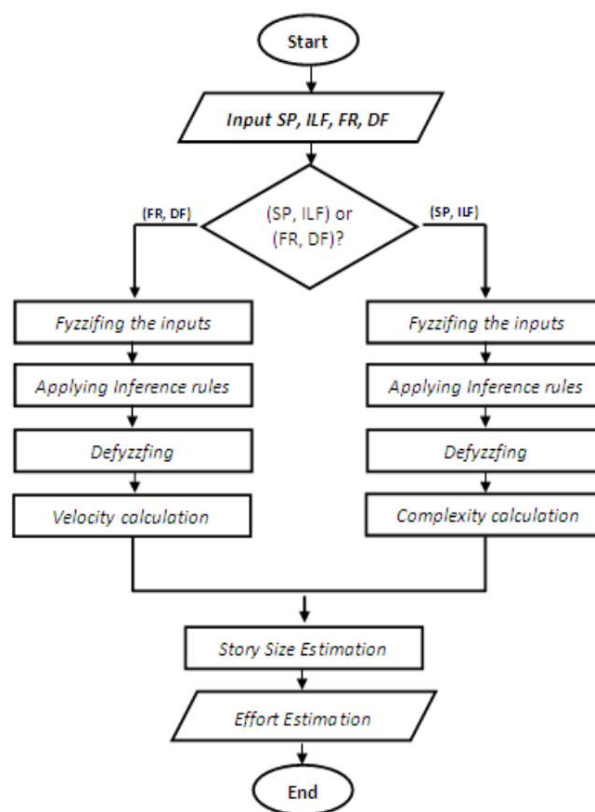


Рисунок 3: Предлагаемая структурная схема

На рисунке 4 показана предлагаемая структура оценки трудозатрат в гибком проекте на основе нечёткой логики, состоящая из трёх фаз. Первая фаза относится к входным данным, включающим три входа: SP, ILF и Com. Эти входные переменные преобразуются в нечёткие переменные в процессе фаззификации. В точке описания истории для этих входных данных были определены термины «Очень маленькая история» (VSS), «Малая история» (SS), «Средняя история» (MS), «Большая история» (LS) и «Очень большая история» (VLS).

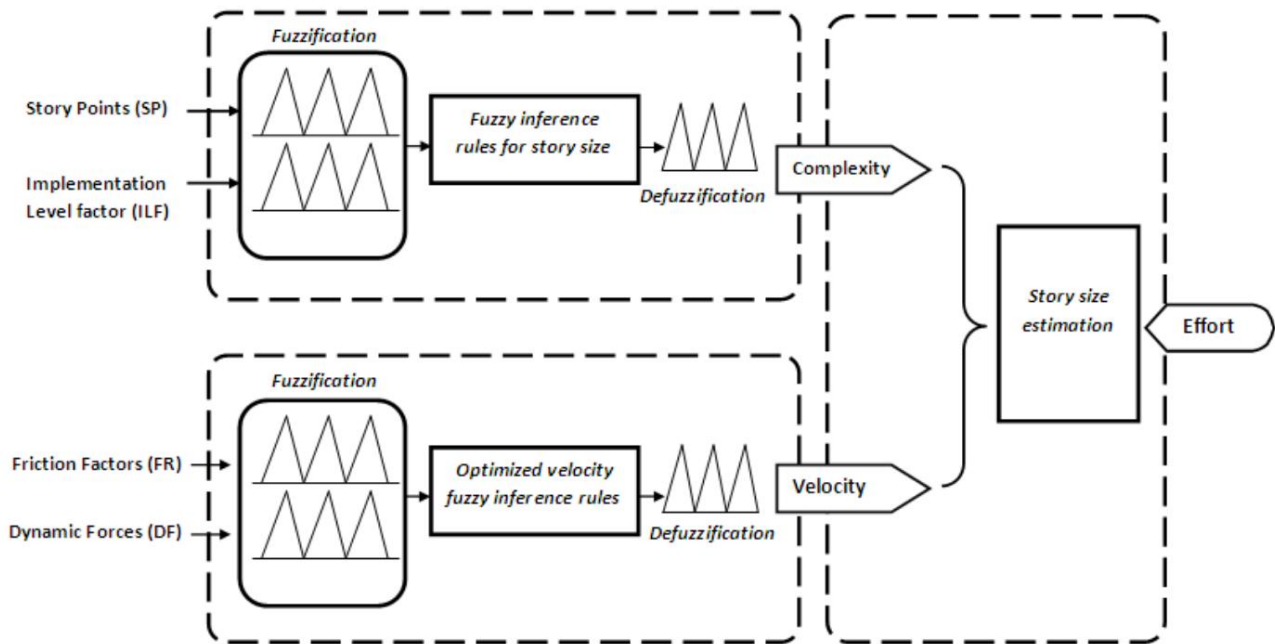


Рисунок 4: Предлагаемая структура для оценки усилий ASD

В таблице 3 показан весовой коэффициент SP, который может иметь множество итераций в зависимости от системных требований. Весовые коэффициенты, назначенные каждой пользовательской истории, прогнозируются с помощью последовательности Фибоначчи.

ТАБЛИЦА 3: BECA SP

Срок	Уайт
Очень маленькая история (VSS)	1
Маленькая история (SS)	2
Средний рассказ (MS)	3
Большая история (LS)	5
Очень большая история (VLS)	8

На рисунке 5 показаны лингвистические переменные для точки повествования, который имеет диапазон веса (VSS, SS, MS, LS и VLS).

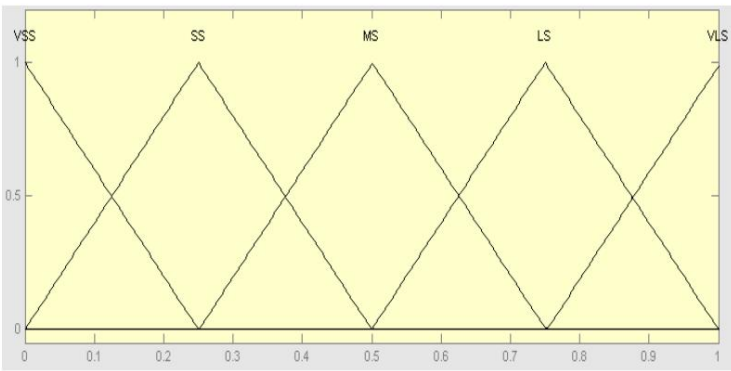


Рисунок 5: Лингвистические переменные для SP

В ILF включены термины «Off the Shelf» (OS), «Full Experience Components» (FEC), «Partial Experience Components» (PEC) и «New Components» (NC). Таблица 4 показывает коэффициент масштабирования для уровня реализации, который отражает уровень понимания каждой пользовательской истории.

ТАБЛИЦА 4: МАСШТАБИРОВАНИЕ ILF

Срок	Шкала
Готовая к использованию (OS)	1
Компоненты полного опыта (FEC)	2
Компоненты частичного опыта (PEC)	3
Новые компоненты (NC)	4

На рисунке 6 показаны лингвистические переменные для реализации, уровень, имеющий диапазон веса (OS, FEC, PEC и NC).

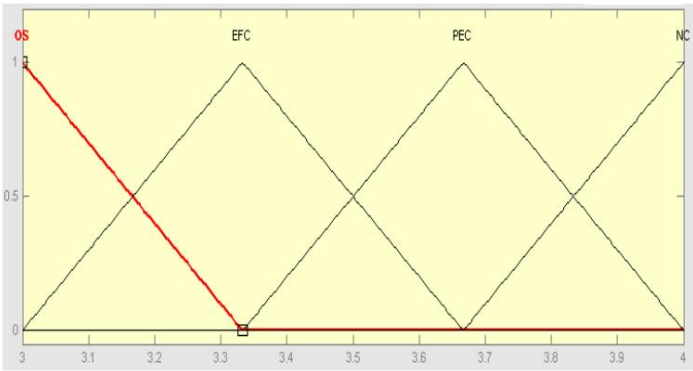


Рисунок 6: Лингвистические переменные для ILF

В таблице 5 уровень сложности Com представлен в категориях «Очень низкий» (VL), «Низкий» (L), «Средний» (M), «Высокий» (H) и «Очень высокий» (VH). Сложность определяется на основе пользовательских историй в ILF. Например, если пользовательская история «Очень низкий», а ILF — «ОС», тогда Com очень низкий.

ТАБЛИЦА 5: УРОВНИ COM

Срок	Уровень
Очень маленькая история (VSS)	1
Маленькая история (SS)	2
Средний рассказ (MS)	3
Большая история (LS)	5
Очень большая история (VLS)	8

На рисунке 7 показаны лингвистические переменные для сложности который имеет диапазон веса (VL, L, M, H и VH).

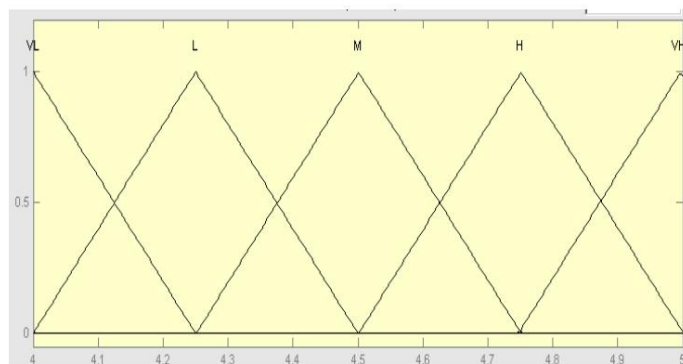


Рисунок 7: Лингвистические переменные для Com

Трудозатраты на весь проект будут представлять собой сумму степеней принадлежности SP, ILF и Com для всех индивидуальных пользовательских историй. Уравнение (2) показывает размер нечёткой истории (FSS) с помощью треугольной функции принадлежности μ .

$$FSS = \sum_{i=1}^4 \mu_{SP_i} \times \sum_{j=1}^5 \mu_{ILF_j} \times \sum_{k=1}^5 \mu_{Com_k} \quad (2)$$

FR включает термины «Стабильный» (S), «Нестабильный» (V), «Высоковолатильно» (HV) и «Очень высоковолатильный» (VHV), которые были определены для четырёх переменных: состава команды, процесса, факторов внешней среды и динамики команды. На рисунке 8 показаны лингвистические переменные для состава команды, имеющие диапазон весов (S, V, HV и VHV).

DF включает термины «Нормальный» (N), «Высокий» (B), «Очень высокий» (OB) и «Экстра высокий» (OVB). Они были определены для девяти переменных: ожидаемая смена команды, внедрение новых инструментов, дефект поставщика, ответственность члена команды вне проекта, личные проблемы, ожидаемая задержка в ответе заинтересованных сторон, ожидаемая неопределенность в деталях, ожидаемые изменения в окружающей среде и ожидаемое перемещение. На рисунке 9 показано

лингвистические переменные для ожидаемых изменений в команде, имеющие диапазон веса (N, H, VH и EH).

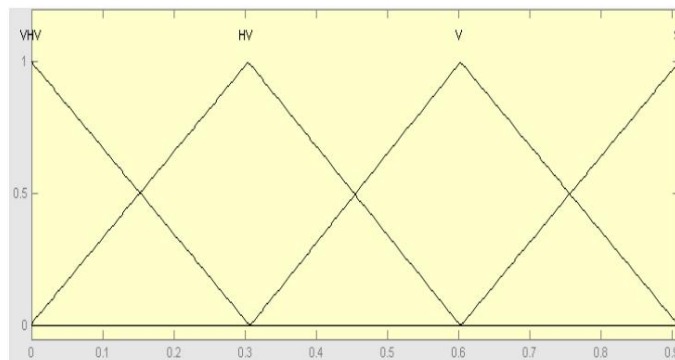


Рисунок 8: Лингвистические переменные для FR

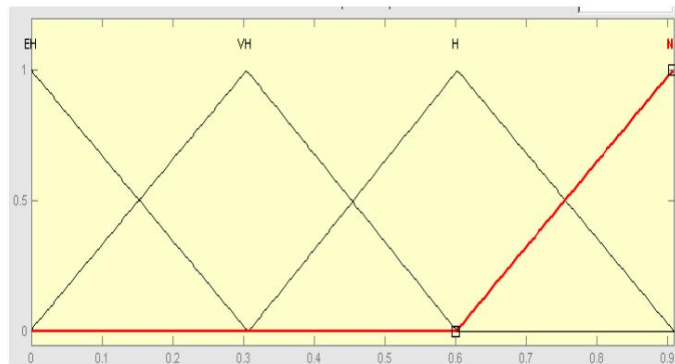


Рисунок 9: Лингвистические переменные для DF

Нечеткий FR (FFR) рассчитывается как произведение всех четырех Компоненты FR и степень принадлежности. Уравнение 3 показывает расчёт FFR с использованием треугольной функции принадлежности μ :

$$FFR = \prod_{i=1}^4 \mu_i \quad (3)$$

Замедление (D) представляет собой скорость отрицательного изменения скорости. Замедление (D) и скорректированная скорость (V) рассчитываются по формуле [33]:

$$D = \Phi P \times D\Phi \quad (4)$$

$$V = (VI) \quad (5)$$

Время компиляции (T) рассчитывается по следующей формуле [33]:

$$T = \frac{\exists}{B} \quad (6)$$

Вторая фаза — это механизм вывода, работающий на основе набора нечётких правил. Правила нечёткой модели содержат лингвистические переменные, относящиеся к гибкому проекту. Ниже представлен пример правил, представляющих предлагаемую структуру:

ЕСЛИ SP это VSS и ILF это OS, TO Com это VL
ЕСЛИ SP это VLS и ILF это NC, TO Com это VLS
ЕСЛИ SP это MS и ILF это OS, TO Com это SS
ЕСЛИ SP это LS и ILF это PEC, TO Com VLS

Третий этап — процесс дефазификации, в ходе которого нечеткие выходные данные вычисляются и преобразуются в четкую форму данных. Наиболее распространенным методом является метод центра тяжести или его еще называют методом центра площади (COA) [34].

$$COA = \frac{\sum_{i=1}^n \mu_i(X) \cdot X}{\sum_{i=1}^n \mu_i(X)}$$

VII. ЗАКЛЮЧЕНИЕ

Целью данной статьи является предложение нечеткой логики для оценки трудозатрат при гибкой разработке программного обеспечения. Исследователи изучили множество исследований в области оценки трудозатрат при гибкой разработке программного обеспечения, как основанных на нечеткой логике, так и нет. Кроме того, исследователи подробно изучили особенности гибкой разработки программного обеспечения, точек истории и нечеткой логики. Затем исследователи предложили фреймворк, основанный на нечеткой логике, которая получает нечеткие входные параметры точек истории, фактор уровня реализации, факторы трения и динамические силы. Предлагаемый фреймворк начинается с входных данных SP, ILF, FR и DF. Затем эти входные данные подвергаются нечеткой логике для получения нечетких множеств с использованием треугольной функции-члена. База правил включает в себя группу правил ЕСЛИ-ТО, которые определяют нечеткие переменные сложности и скорости. Система нечеткого вывода оценивает все правила базы правил. Результирующее нечеткое множество деактивируется до четкого значения, после чего вычисляются сложность и скорость. Наконец, трудозатраты оцениваются с использованием результирующих сложности и скорости.

Исследователи посчитали, что использование нечёткой логики и сюжетной линии в предлагаемой структуре может помочь в точной оценке трудозатрат. Исследователи разработали предлагаемую структуру с помощью MATLAB, чтобы подготовить её к последующим экспериментам с реальными наборами данных.

VIII. БУДУЩАЯ РАБОТА

Идеи, которым, как ожидается, будет уделяться особое внимание в будущем, включают:

- Применение предлагаемой в данной статье структуры с использованием реальных наборов данных из программных проектов и сравнение результатов с другими результатами, полученными с помощью различных методов.
- Расширение структуры с использованием результатов применения.

- Использование COCOMOII для улучшения предлагаемой структуры

ССЫЛКИ

- [1] Абир Хамди, «Нечеткая логика для повышения чувствительности модели стоимости COCOMO», Журнал новых тенденций в вычислительной технике и информационных науках, том 3, № 9, 2012.
- [2] Амрита Радж Муккер, Анил Кумар Мишра и Латика Сингх, «Повышение качества в проектах по разработке программного обеспечения Scrum», Международный журнал науки и исследований (IJSR), том 3, выпуск 4, 2014.
- [3] Андреас Шмитендорф, Мартин Кунц и Райнер Думке, «Оценка усилий для проектов гибкой разработки программного обеспечения», 5-й Европейский форум по измерению программного обеспечения, 2008 г.
- [4] Энн Л. Фрулинг и Элвин Э. Таррелл, «Лучшие практики внедрения гибких методов», IBM, 2008.
- [5] Анулама Кошик, А.К. Сони и Рахна Сони, «Нечеткая логика типа 2». «Основная структура для функциональных точек»,
- [6] Ашита Малик, Варун Пандей, Анулама Каушик, «Анализ нечетких подходов для COCOMO II», Международный журнал интеллектуальных систем и приложений, том 5, 2013 г.
- [7] Бек и др., «<http://agilemanifesto.org/iso/en/manifesto.html>», последний раз посещали в декабре 2014 года.
- [8] С. Sathish Kumar и R. Srinivasan, «Оптимизированный гибкий план оценки с использованием алгоритма поиска Harmony», Международный журнал инжиниринга и технологий (IJET), том 6, № 5, 2014.
- [9] Даниэле Ди Филиппо, Рассел Д. Арчибальд и Ивано Ди Филиппо, «Шестиэтапная комплексная модель жизненного цикла проекта», том I, выпуск V, декабрь 2012 г.
- [10] Дэвид Козн, Микаэль Линдвалл, Патрисия Коста, «Введение в гибкие методы», Достижения в области компьютеров, Elsevier, т. 62, стр. 20-22, 2004.
- [11] Эвита Козльо, Анирбан Басу, «Оценка усилий при гибкой разработке программного обеспечения с использованием Story Points», Международный журнал прикладных информационных систем (IJASIS), том 3- № 7, август 2012 г.
- [12] Ганеш М., «Введение в нечеткие множества и нечеткую логику», Prentice-Hall, 978-8120328617, 2006.
- [13] Иоаннис Г. Стамелос и Панайотис Сфетсос, «Гибкая разработка программного обеспечения: обеспечение качества», Idea Group, ISBN 978-1-59904-216-9, 2007.
- [14] Джейсон Уэстленд, «Руководство по управлению проектами», Method123, 2003.
- [15] Джефф Дайер, Джеффри Х. Дайер, Уильям Г., «Создание команды: проверенные стратегии повышения эффективности команды», Jossey-Bass, ISBN: 978-1118416143, 2013.
- [16] Джитендер Чоудхурия и Уграсен Суман, «Модель оценки усилий на основе точек истории для обслуживания программного обеспечения», Elsevier, Procedia Technology, том 4, 2012.
- [17] Джон Хант, «Гибкая разработка программного обеспечения», Springer, ISBN-10: 1-85233-944-6, 2006.
- [18] К.Уша, Н.Пунгузали и Э.Кавита, «Количественный подход к оценке эффективности рефакторинга в процессе разработки программного обеспечения», Международная конференция по методам и моделям в информатике, Дели, Индия, декабрь 2009 г.
- [19] Кевин Вландаерен, Сьяак Бринккемпер, Слинггер Янсен, Эрик Ясперс, «Применение принципов SCRUM к управлению программными продуктами». Информационные и программные технологии, Т. 53, Выпуск 1, стр. 58-70, 2011.
- [20] Комал Гарг, Парамджит Каур, Шалини Капур и Шилпа Нарула, «Улучшение модели COCOMO с использованием анализа точек функции для повышения оценки усилий», IJCSMC, том 3, выпуск 6, 2014.
- [21] Луиджи Бульоне и Ален Абран, «Улучшение оценок в гибких проектах: проблемы и пути», Европейский форум по измерению программного обеспечения (SMEF), 2007.
- [22] MC Ohlsson, C. Wohlin и B. Regnell, «Исследование оценки усилий по проекту», Elsevier Science, информационные и программные технологии, том 40, 831-839, 1998.

- [23] Масатеру Цунода, Кодзи Тода и Кёхеи Фусида, Ясутака Камей, Мейаппан Нагаппан и Наоясу Убаяши, «Пересмотр оценки усилий по разработке программного обеспечения на основе деятельности на ранней стадии разработки», 10-я конференция по репозиториям программного обеспечения для горнодобывающей промышленности (MSR), IEEE, 2013.
- [24] Майк Кон, «Разработка программного обеспечения с использованием Scrum», Addison-Wesley, ISBN: 978-0321579362, 2009.
- [25] Наги Рамадан Дарвиш, «Повышение качества применения подхода экстремального программирования (XP)», Международный журнал компьютерных наук и информационной безопасности (IJCSIS) – ISSN 1947-5500, том 9, № 11, ноябрь 2011 г.
- [26] Ратнеш Литория и Абхай Котари «Эффективный подход к оценке гибких веб-проектов: AgileMOW», Журнал по программной инженерии и приложениям, том 6, 2013.
- [27] Сандип Кад, Винай Чопра, «Основанная на нечеткой логике структура для оценки усилий по разработке программного обеспечения», Международный журнал инженерных наук (IJES), том 1, 2011.
- [28] Шива Дорайрадж, Джеймс Ноубл, и Петра Малик, «Понимание динамики команды при распределенной гибкой разработке программного обеспечения», Springer, ISBN: 978-3-642-30350-0, 2012.
- [29] Варста Дж., Ронкайнен Дж., Сало О., Абрахамссон П., «гибкое программное обеспечение», BTT, ISBN 951-38-6009-4, 2002.
- [30] Вишал Шарма и Харш Кумар Верма, «Оптимизированная структура на основе нечеткой логики для оценки усилий при разработке программного обеспечения», Международный журнал по проблемам компьютерной науки (IJCSI), том 7, выпуск 2, № 2, 2010.
- [31] Виталий Зуриан, «<http://www.agilemarketing.net/winning-planning-poker/>», последний раз посетил сайт в июне 2015 года.
- [32] Ян Юн и Бошэн Чжоу, «Оценка эффекта экстремального программирования с помощью моделирования системной динамики», Международная конференция по вычислительному интеллекту и программной инженерии (CiSE), Ухань, Китай, декабрь 2009 г.
- [33] Зиауддин Зия, Шахид Камал Типу и Шахрух Зия «Модель оценки усилий для гибкой разработки программного обеспечения», Достижения в области компьютерной науки и ее приложений (ACSA), том 2, 2012.
- [34] Зиауддин, Шахид Камал, Шафиуллах Хан и Джамал Абдул Насир, «Модель оценки стоимости программного обеспечения на основе нечеткой логики», Международный журнал по программной инженерии и ее применению (IJSEIA), том 7, выпуск 2, 2013 г.